AV1 Image File Format (AVIF)

v1.2.0

AOM Final Deliverable, 16 October 2025

This version:
https://aomediacodec.github.io/av1-avif/v1.2.0.html

Latest approved version:
https://aomediacodec.github.io/av1-avif/latest-approved.html

Latest version (published or draft):
https://aomediacodec.github.io/av1-avif/index.html

Previously approved version:
https://aomediacodec.github.io/av1-avif/v1.1.0.html

Issue Tracking:
GitHub

Editors:
Yannis Guyon (Google)
Leo Barnes (Apple)
Wan-Teh Chang (Google)

Former Editors:
Cyril Concolato (Netflix)
Paul Kerr (Netflix)
Anders Klemets (Microsoft)

Copyright 2025, AOM

Licensing information is available at http://aomedia.org/license/

The MATERIALS ARE PROVIDED “AS IS.” The Alliance for Open Media, its members, and its contributors expressly disclaim any warranties (express, implied, or otherwise), including implied warranties of merchantability, non-infringement, fitness for a particular purpose, or title, related to the materials. The entire risk as to implementing or otherwise using the materials is assumed by the implementer and user. IN NO EVENT WILL THE ALLIANCE FOR OPEN MEDIA, ITS MEMBERS, OR CONTRIBUTORS BE LIABLE TO ANY OTHER PARTY FOR LOST PROFITS OR ANY FORM OF INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS DELIVERABLE OR ITS GOVERNING AGREEMENT, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND WHETHER OR NOT THE OTHER MEMBER HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Abstract

This document specifies syntax and semantics for the storage of [AV1] images in the generic image file format [HEIF], which is based on [ISOBMFF]. While [HEIF] defines general requirements, this document also specifies additional constraints to ensure higher interoperability between writers and readers when [HEIF] is used with [AV1] images. These constraints are based on constraints defined in the Multi-Image Application Format [MIAF] and are grouped into profiles inspired by the profiles defined in [MIAF].

Table of Contents

1 Scope

2 Image Items and properties
2.1 AV1 Image Item
2.2 Image Item Properties
  2.2.1 AV1 Item Configuration Property
  2.2.2 Image Spatial Extents Property
  2.2.3 Clean Aperture Property
  2.2.4 Other Item Properties
2.3 AV1 Layered Image Items
  2.3.1 Overview
  2.3.2 Properties
    2.3.2.1 Operating Point Selector Property
    2.3.2.2 Layer Selector Property
    2.3.2.3 Layered Image Indexing Property

3 Image Sequences

4 Other Image Items and Sequences
4.1 Auxiliary Image Items and Sequences
4.2 Derived Image Items
  4.2.1 Grid Derived Image Item
  4.2.2 Tone Map Derived Image Item
  4.2.3 Sample Transform Derived Image Item

5 Entity groups
5.1 'altr' group
5.2 'ster' group

6 Brands, Internet media types and file extensions
6.1 Brands overview
6.2 AVIF image and image collection brand
6.3 AVIF image sequence brands

7 General constraints

8 Profiles
8.1 Overview
8.2 AVIF Baseline Profile
8.3 AVIF Advanced Profile

9 Box requirements
9.1 Image item boxes
  9.1.1 Minimum set of boxes
  9.1.2 Requirements on additional image item related boxes

10 AVIF Media Type Registration

11 Changes since v1.1.0 release

Appendix A: (informative) Sample Transform Derived Image Item Examples
Bit depth extension
  Suffix bit depth extension
  Residual bit depth extension

Conformance

Index
Terms defined by this specification
Terms defined by reference

References
Normative References

1. Scope

[AV1] defines the syntax and semantics of an AV1 bitstream. The ***AV1 Image File Format*** (***AVIF***) defined in this document supports the storage of a subset of the syntax and semantics of an AV1 bitstream in a [HEIF] file. The AV1 Image File Format defines multiple profiles, which restrict the allowed syntax and semantics of the AV1 bitstream with the goal to improve interoperability, especially for hardware implementations. The profiles defined in this specification follow the conventions of the [MIAF] specification. Images encoded with [AV1] and not meeting the restrictions of the defined profiles may still be compliant to this AV1 Image File Format if they adhere to the general AVIF requirements.

The AV1 Image File Format supports High Dynamic Range (HDR) and Wide Color Gamut (WCG) images as well as Standard Dynamic Range (SDR). It supports monochrome images as well as multi-channel images with all the bit depths and color spaces specified in [AV1], and other bit depths with Sample Transform Derived Image Items. The AV1 Image File Format also supports transparency (alpha) and other types of data such as depth maps through auxiliary AV1 bitstreams.

The AV1 Image File Format also supports multi-layer images as specified in [AV1] to be stored both in image items and image sequences. The AV1 Image File Format supports progressive image decoding through layered images.

An ***AVIF file*** is designed to be a conformant [HEIF] file for both image items and image sequences. Specifically, this specification follows the recommendations given in "Annex I: Guidelines On Defining New Image Formats and Brands" of [HEIF].

This specification reuses syntax and semantics used in [AV1-ISOBMFF].

2. Image Items and properties

2.1. AV1 Image Item

When an item is of type ***av01***, it is called an ***AV1 Image Item***, and shall obey the following constraints:

- The AV1 Image Item shall be a conformant MIAF image item.
- The AV1 Image Item shall be associated with an `AV1ItemConfigurationProperty`.
- The content of an AV1 Image Item is called the ***AV1 Image Item Data*** and shall obey the following constraints:
  - The AV1 Image Item Data shall be identical to the content of an AV1 Sample marked as `'sync'`, as defined in [AV1-ISOBMFF].
  - The AV1 Image Item Data shall have exactly one Sequence Header OBU.

  > NOTE: File writers may want to set the `still_picture` and `reduced_still_picture_header` flags to 1 when possible in the Sequence Header OBU part of the AV1 Image Item Data so that AV1 header overhead is minimized.

2.2. Image Item Properties

2.2.1. AV1 Item Configuration Property

```
Box Type:                av1C
Property type:           Descriptive item property
Container:               ItemPropertyContainerBox
Mandatory (per item):    Yes, for an image item of type 'av01', no otherwise
Quantity (per item):     One for an image item of type 'av01', zero otherwise
```

The syntax and semantics of the ***AV1ItemConfigurationProperty*** are identical to those of the `AV1CodecConfigurationBox` defined in [AV1-ISOBMFF], with the following constraints:

- Sequence Header OBUs should not be present in the `AV1ItemConfigurationProperty`.
- If a Sequence Header OBU is present in the `AV1ItemConfigurationProperty`, it shall match the Sequence Header OBU in the AV1 Image Item Data.
- The values of the fields in the `AV1ItemConfigurationProperty` shall match those of the Sequence Header OBU in the AV1 Image Item Data.
- The values of the bit depth and the number of channels derived from the `AV1ItemConfigurationProperty` shall match the `PixelInformationProperty` (`'pixi'`) if present.
- Metadata OBUs, if present, shall match the values given in other item properties, such as the `MasteringDisplayColourVolumeBox` (`'mdcv'`) or `ContentLightLevelBox` (`'clli'`).

This property should be marked as essential.

2.2.2. Image Spatial Extents Property

The semantics of the `'ispe'` property as defined in [HEIF] apply. More specifically, for [AV1] images, the values of `image_width` and `image_height` shall respectively equal the values of `UpscaledWidth` and `FrameHeight` as defined in [AV1] but for a specific frame in the item payload. The exact frame depends on the presence and content of the `'lsel'` and `OperatingPointSelectorProperty` properties as follows:

- In the absence of a `'lsel'` property associated with the item, or if it is present and its `layer_id` value is set to 0xFFFF:
  - If no `OperatingPointSelectorProperty` is associated with the item, the `'ispe'` property shall document the dimensions of the last frame decoded when processing the operating point whose index is 0.
  - If an `OperatingPointSelectorProperty` is associated with the item, the `'ispe'` property shall document the dimensions of the last frame decoded when processing the corresponding operating point.

  > NOTE: The dimensions of possible intermediate output images might not match the ones given in the `'ispe'` property. If renderers display these intermediate images, they are expected to scale the output image to match the `'ispe'` property.
- If a `'lsel'` property is associated with an item and its `layer_id` is different from 0xFFFF, the `'ispe'` property documents the dimensions of the output frame produced by decoding the corresponding layer.

> NOTE: The dimensions indicated in the `'ispe'` property might not match the values `max_frame_width_minus1`+1 and `max_frame_height_minus1`+1 indicated in the AV1 bitstream.

> NOTE: The values of `render_width_minus1` and `render_height_minus1` possibly present in the AV1 bitstream are not exposed at the AVIF container level.

2.2.3. Clean Aperture Property

The semantics of the clean aperture property (`'clap'`) as defined in [HEIF] apply. In addition to the restrictions on transformative item property ordering specified in [MIAF], the following restriction also applies:

The origin of the `'clap'` item property shall be anchored to 0,0 (top-left) of the input image unless the full, uncropped image item is included as a secondary non-hidden image item.

2.2.4. Other Item Properties

In addition to the Image Properties defined in this document, AV1 image items may also be associated with item properties defined in other specifications such as [HEIF] and [MIAF]. Commonly used item properties can be found in §9.1.1 Minimum set of boxes and §9.1.2 Requirements on additional image item related boxes.

In general, it is recommended to use item properties instead of Metadata OBUs in the `AV1ItemConfigurationProperty`.

2.3. AV1 Layered Image Items

2.3.1. Overview

[AV1] supports encoding a frame using multiple spatial layers. A spatial layer may improve the resolution or quality of the image decoded based on one or more of the previous layers. A layer may also provide an image that does not depend on the previous layers. Additionally, not all layers are expected to produce an image meant to be rendered. Some decoded images may be used only as intermediate decodes. Finally, layers are grouped into one or more Operating Points. The Sequence Header OBU defines the list of Operating Points, provides required decoding capabilities, and indicates which layers form each Operating Point.

[AV1] delegates the selection of which Operating Point to process to the application, by means of a function called `choose_operating_point()`. AVIF defines the `OperatingPointSelectorProperty` to control this selection. In the absence of an `OperatingPointSelectorProperty` associated with an AV1 Image Item, the AVIF renderer is free to process any Operating Point present in the AV1 Image Item Data. In particular, when the AV1 Image Item is composed of a unique Operating Point, the `OperatingPointSelectorProperty` should not be present. If an `OperatingPointSelectorProperty` is associated with an AV1 Image Item, the `op_index` field indicates which Operating Point is expected to be processed for this item.

> NOTE: When an author wants to offer the ability to render multiple Operating Points from the same AV1 image (e.g. in the case of multi-view images), multiple AV1 Image Items can be created that share the same AV1 Image Item Data but have different `OperatingPointSelectorProperties`.

[AV1] expects the renderer to display only one frame within the selected Operating Point, which should be the highest spatial layer that is both within the Operating Point and present within the temporal unit, but [AV1] leaves the option for other applications to set their own policy about which frames are output, as defined in the general output process. AVIF sets a different policy, and defines how the `'lsel'` property (mandated by [HEIF] for layered images) is used to control which layer is rendered. According to [HEIF], the interpretation of the `layer_id` field in the `'lsel'` property is codec specific. In this specification, the value 0xFFFF is reserved for a special meaning. If a `'lsel'` property is associated with an AV1 Image Item but its `layer_id` value is set to 0xFFFF, the renderer is free to render either only the output image of the highest spatial layer, or to render all output images of all the intermediate layers and the highest spatial layer, resulting in a form of progressive decoding. If a `'lsel'` property is associated with an AV1 Image Item and the value of `layer_id` is not 0xFFFF, the renderer is expected to render only the output image for that layer.

> NOTE: When such a progressive decoding of the layers within an Operating Point is not desired or when an author wants to expose each layer as a specific item, multiple AV1 Image Items sharing the same AV1 Image Item Data can be created and associated with different `'lsel'` properties, each with a different value of `layer_id`.

2.3.2. Properties

2.3.2.1. Operating Point Selector Property

2.3.2.1.1. DEFINITION

```
Box Type:                a1op
Property type:           Descriptive item property
Container:               ItemPropertyContainerBox
Mandatory (per item):    No
Quantity (per item):     Zero or one
```

2.3.2.1.2. DESCRIPTION

An ***OperatingPointSelectorProperty*** may be associated with an AV1 Image Item to provide the index of the operating point to be processed for this item. If associated, it shall be marked as essential.

2.3.2.1.3. SYNTAX

```
class OperatingPointSelectorProperty extends ItemProperty('a1op') {
    unsigned int(8) op_index;
}
```

2.3.2.1.4. SEMANTICS

op_index indicates the index of the operating point to be processed for this item. Its value shall be between 0 and `operating_points_cnt_minus_1` inclusive.

2.3.2.2. Layer Selector Property

The `'lsel'` property defined in [HEIF] may be associated with an AV1 Image Item. The `layer_id` indicates the value of the `spatial_id` to render. The value shall be between 0 and 3, or the special value 0xFFFF. When a value between 0 and 3 is used, the corresponding spatial layer shall be present in the bitstream and shall produce an output frame. Other layers may be needed to decode the indicated layer. When the special value 0xFFFF is used, progressive decoding is allowed as described in §2.3.1 Overview.

2.3.2.3. Layered Image Indexing Property

2.3.2.3.1. DEFINITION

```
Box Type:                a1lx
Property type:           Descriptive item property
Container:               ItemPropertyContainerBox
Mandatory (per item):    No
Quantity (per item):     Zero or one
```

2.3.2.3.2. DESCRIPTION

The ***AV1LayeredImageIndexingProperty*** property may be associated with an AV1 Image Item. It should not be associated with AV1 Image Items consisting of only one layer.

The `AV1LayeredImageIndexingProperty` documents the size in bytes of each layer (except the last one) in the AV1 Image Item Data, and enables determining the byte ranges required to process one or more layers of an Operating Point. If associated, it shall not be marked as essential.

2.3.2.3.3. SYNTAX

```
class AV1LayeredImageIndexingProperty extends ItemProperty('a1lx') {
    unsigned int(7) reserved = 0;
    unsigned int(1) large_size;
    FieldLength = (large_size + 1) * 16;
    unsigned int(FieldLength) layer_size[3];
}
```

2.3.2.3.4. SEMANTICS

layer_size indicates the number of bytes corresponding to each layer in the item payload, except for the last layer. Values are provided in increasing order of `spatial_id`. A value of zero means that all the layers except the last one have been documented and following values shall be 0. The number of non-zero values shall match the number of layers in the image minus one.

> NOTE: The size of the last layer can be determined by subtracting the sum of the sizes of all layers indicated in this property from the entire item size.

> EXAMPLE 1 ¶
> A property indicating [X,0,0] is used for an image item composed of 2 layers. The size of the first layer is X and the size of the second layer is ItemSize - X. Note that the `spatial_id` for the first layer does not necessarily match the index in the array that provides the size. In other words, in this case the index giving value X is 0, but the corresponding `spatial_id` could be 0, 1 or 2. Similarly, a property indicating [X,Y,0] is used for an image made of 3 layers.

3. Image Sequences

An ***AV1 Image Sequence*** is defined as a set of AV1 Temporal Units stored in an AV1 track as defined in [AV1-ISOBMFF] with the following constraints:

- The track shall be a valid MIAF image sequence.
- The track handler for an AV1 Image Sequence shall be `'pict'`.
- The track shall have only one AV1 Sample description entry.
- If multiple Sequence Header OBUs are present in the track payload, they shall be identical.

4. Other Image Items and Sequences

4.1. Auxiliary Image Items and Sequences

An ***AV1 Auxiliary Image Item*** (respectively an ***AV1 Auxiliary Image Sequence***) is an AV1 Image Item (respectively AV1 Image Sequence) with the following additional constraints:

- It shall be a compliant MIAF Auxiliary Image Item (respectively MIAF Auxiliary Image Sequence).
- The `mono_chrome` field in the Sequence Header OBU shall be set to 1.
- The `color_range` field in the Sequence Header OBU shall be set to 1.

An ***AV1 Alpha Image Item*** (respectively an ***AV1 Alpha Image Sequence***) is an AV1 Auxiliary Image Item (respectively an AV1 Auxiliary Image Sequence), and as defined in [MIAF], with the `aux_type` field of the `AuxiliaryTypeProperty` (respectively `AuxiliaryTypeInfoBox`) set to `urn:mpeg:mpegB:cicp:systems:auxiliary:alpha`. An AV1 Alpha Image Item (respectively an AV1 Alpha Image Sequence) shall be encoded with the same bit depth as the associated master AV1 Image Item (respectively AV1 Image Sequence).

For AV1 Alpha Image Items and AV1 Alpha Image Sequences, the `ColourInformationBox` (`'colr'`) should be omitted. If present, readers shall ignore it.

An ***AV1 Depth Image Item*** (respectively an ***AV1 Depth Image Sequence***) is an AV1 Auxiliary Image Item (respectively an AV1 Auxiliary Image Sequence), and as defined in [MIAF], with the `aux_type` field of the `AuxiliaryTypeProperty` (respectively `AuxiliaryTypeInfoBox`) set to `urn:mpeg:mpegB:cicp:systems:auxiliary:depth`.

> NOTE: [AV1] supports encoding either 3-component images (whose semantics are given by the `matrix_coefficients` element), or 1-component images (monochrome). When an image requires a different number of components, multiple auxiliary images may be used, each providing additional component(s), according to the semantics of their `aux_type` field. In such case, the maximum number of components is restricted by number of possible items in a file, coded on 16 or 32 bits.

4.2. Derived Image Items

4.2.1. Grid Derived Image Item

A ***grid derived image item*** (`'grid'`) as defined in [HEIF] may be used in an AVIF file.

4.2.2. Tone Map Derived Image Item

A ***tone map derived image item*** (`'tmap'`) as defined in [HEIF] may be used in an AVIF file. When present, the base image item and the `'tmap'` image item should be grouped together by an `'altr'` (see §5.1 'altr' group) entity group as recommended in [HEIF]. When present, the gain map image item should be a hidden image item.

4.2.3. Sample Transform Derived Image Item

With a ***Sample Transform Derived Image Item***, pixels at the same position in multiple input image items can be combined into a single output pixel using basic mathematical operations. This can for example be used to work around codec limitations or for storing alterations to an image as non-destructive residuals. With a Sample Transform Derived Image Item it is possible for AVIF to support 16 or more bits of precision per sample, while still offering backward compatibility through a regular 8 to 12-bit AV1 Image Item containing the most significant bits of each sample.

In these sections, a "sample" refers to the value of a pixel for a given channel.

4.2.3.1. Definition

When a derived image item is of type `'sato'`, it is called a Sample Transform Derived Image Item, and its reconstructed image is formed from a set of input image items, constants and operators.

The input images are specified in the `SingleItemTypeReferenceBox` or `SingleItemTypeReferenceBoxLarge` entries of type `'dimg'` for this Sample Transform Derived Image Item within the `ItemReferenceBox`. The input images are in the same order as specified in these entries. In the `SingleItemTypeReferenceBox` or `SingleItemTypeReferenceBoxLarge` of type `'dimg'`, the value of the `from_item_ID` field identifies the Sample Transform Derived Image Item, and the values of the `to_item_ID` field identify the input images. There are `reference_count` input image items as specified by the `ItemReferenceBox`.

The input image items and the Sample Transform Derived Image Item shall:

- each be associated with a `PixelInformationProperty` and an `'ispe'` property;
- have the same number of channels as defined by the `PixelInformationProperty` or `AV1ItemConfigurationProperty`;
- have the same chroma subsampling; this may be explicitly defined by one of the above properties for some input image items or the Sample Transform Derived Image Item, and may be implicit for the other items (meaning no property defines chroma subsampling for these items);
- have the same dimensions as defined by the `'ispe'` property;
- have the same color information as defined by the `ColourInformationBox` properties (or lack thereof).

Each output sample of the Sample Transform Derived Image Item is obtained by evaluating an expression consisting of a series of integer operators and operands. An operand is a constant or a sample from an input image item located at the same channel index and at the same spatial coordinates as the output sample.

No color space conversion, matrix coefficients, or transfer characteristics function shall be applied to the input samples. They are already in the same color space as the output samples.

The output reconstructed image is made up of the output samples, whose values shall each be clamped to fit in the number of bits per sample as defined by the `PixelInformationProperty` of the reconstructed image item. The `full_range_flag` field of the `ColourInformationBox` property of `colour_type` `'nclx'` also defines a range of values to clamp to, as defined in [CICP].

> NOTE: Appendix A: (informative) Sample Transform Derived Image Item Examples contains examples of Sample Transform Derived Image Item usage.

4.2.3.2. Syntax

An ***expression*** is a series of tokens. A token is an operand or an operator. An operand can be a literal constant value or a sample value. A stack is used to keep track of the results of the subexpressions. An operator takes either one or two input operands. Each unary operator pops one value from the stack. Each binary operator pops two values from the stack, the first being the right operand and the second being the left operand. Each token results in a value pushed to the stack. The single remaining value in the stack after evaluating the whole expression is the resulting output sample.

```
aligned(8) class SampleTransform {
    unsigned int(2) version = 0;
    unsigned int(4) reserved;
    unsigned int(2) bit_depth; // Enum signaling signed 8, 16, 32 or 64-bit.
    // Create an empty stack of signed integer elements of that depth.
    unsigned int(8) token_count;
    for (i=0; i<token_count; i++) {
        unsigned int(8) token;
        if (token == 0) {
            // Push the 'constant' value to the stack.
            signed int(1<<(bit_depth+3)) constant;
        } else if (token <= 32) {
            // Push the sample value from the 'token'th input image item
            // to the stack.
        } else {
            if (token >= 64 && token <= 67) {
                // Unary operator. Pop the operand from the stack.
            } else if (token >= 128 && token <= 137) {
                // Binary operator. Pop the right operand
                // and then the left operand from the stack.
            }
            // Apply operator 'token' and push the result to the stack.
        }
    }
    // Output the single remaining stack element.
}
```

4.2.3.3. Semantics

version shall be equal to 0. Readers shall ignore a Sample Transform Derived Image Item with an unrecognized `version` number.

reserved shall be equal to 0. The value of `reserved` shall be ignored by readers.

bit_depth determines the precision (from 8 to 64 bits, see Table 1) of the signed integer temporary variable supporting the intermediate results of the operations. It also determines the precision of the stack elements and the field size of the `constant` fields. This intermediate precision shall be high enough so that all input sample values fit into that signed bit depth.

Value of `bit_depth`	Intermediate bit depth (sign bit inclusive) `num_bits`
0	8
1	16
2	32
3	64

*Table 1 - Mapping from `bit_depth` to the **intermediate bit depth** (`num_bits`).*

The result of any computation underflowing or overflowing the intermediate bit depth is replaced by -2^{num_bits-1} and $2^{num\_bits-1}-1$, respectively. Encoder implementations should not create files leading to potential computation underflow or overflow. Decoder implementations shall check for computation underflow or overflow and clamp the results accordingly. Computations with operands of negative values use the two’s-complement representation.

token_count is the expected number of tokens to read. The value of `token_count` shall be greater than 0.

token determines the type of the ***operand*** (`constant` or input image item sample) or the ***operator*** (how to transform one or two operands into the result). See Table 2. Readers shall ignore a Sample Transform Derived Image Item with a reserved `token` value.

Value of `token`	Token name	Token type	Meaning before pushing to the stack	Value pushed to the stack (L and R refer to operands popped from the stack for operators)
0	constant	operand	$2^{bit\_depth+3}$ bits from the stream read as a signed integer.	constant value
1..32	sample	operand	Sample value from the `token`th input image item (`token` is the 1-based index of the input image item whose sample is pushed to the stack).	input image item sample value
33..63			*Reserved*	
64	negation	unary operator	Negation of the left operand.	$-L$
65	absolute value	unary operator	Absolute value of the left operand.	$\lvert L \rvert$
66	not	unary operator	Bitwise complement of the operand.	$\neg L$
67	bsr	unary operator	0-based index of the most significant set bit of the left operand if the left operand is strictly positive, zero otherwise.	$\begin{cases} 0 & \text{if } L \le 0 \\ (\log_2 L) & \end{cases}$
68..127			*Reserved*	
128	sum	binary operator	Left operand added to the right operand.	$L + R$
129	difference	binary operator	Right operand subtracted from the left operand.	$L - R$
130	product	binary operator	Left operand multiplied by the right operand.	$L \times R$
131	quotient	binary operator	Left operand divided by the right operand if the right operand is not zero, left operand otherwise. The result is truncated toward zero (integer division).	$\begin{cases} L & \text{if } R = 0 \\ (\frac{L}{R}) & \end{cases}$
132	and	binary operator	Bitwise conjunction of the operands.	$L \wedge R$
133	or	binary operator	Bitwise inclusive disjunction of the operands.	$L \vee R$
134	xor	binary operator	Bitwise exclusive disjunction of the operands.	$L \oplus R$

135	pow	binary operator	Left operand raised to the power of the right operand if the left operand is not zero, zero otherwise.	$\begin{cases} 0 & \text{if } L = 0 \\ (L^R) & \end{cases}$
136	min	binary operator	Minimum value among the operands.	$\begin{cases} L & \text{if } L \leq R \\ R & \end{cases}$
137	max	binary operator	Maximum value among the operands.	$\begin{cases} R & \text{if } L \leq R \\ L & \end{cases}$
138..255			*Reserved*	

Table 2 - Meaning of the value of `token`.

constant is a literal signed value extracted from the stream with a precision of intermediate bit depth, pushed to the stack.

§ *4.2.3.4. Constraints*

Sample Transform Derived Image Items use the postfix notation to evaluate the result of the whole expression for each reconstructed image item sample.

- The tokens shall be evaluated in the order they are defined in the metadata (the ***`SampleTransform`*** structure defined in §4.2.3.2 Syntax) of the Sample Transform Derived Image Item.
- `token` shall be at most `reference_count` when evaluating a sample operand (when $1 \leq \text{token} \leq 32$).
- There shall be at least one `token`.
- The stack is empty before evaluating the first `token`.
- There shall be at least 1 element in the stack before evaluating a unary operator.
- There shall be at least 2 elements in the stack before evaluating a binary operator.
- There shall be exactly one remaining element in the stack after evaluating the last `token`. This element is the value of the reconstructed image item sample.

Non-compliant expressions shall be rejected by parsers as invalid files.

> NOTE: Because each operator pops one or two elements and then pushes one element to the stack, there is at most one more operand than operators in the expression. There are at least $\text{floor}\left(\frac{\text{token\_count}}{2}\right)$ operators and at most $\text{ceil}\left(\frac{\text{token\_count}}{2}\right)$ operands. `token_count` is at most 255, meaning the maximum stack size for a valid expression is 128.

§ 5. Entity groups

The `GroupsListBox` ('`grpl`') defined in [ISOBMFF] may be used to group multiple image items or tracks in a file together. The type of the group describes how the image items or tracks are related. Decoders should ignore groups of unknown type.

§ 5.1. '`altr`' group

The '`altr`' entity group as defined in [ISOBMFF] may be used to mark multiple items or tracks as alternatives to each other. Only one item or track in the '`altr`' group should be played or processed. This grouping is useful for defining a fallback for parsers when new types of items or essential item properties are introduced.

§ 5.2. '`ster`' group

The '`ster`' entity group as defined in [HEIF] may be used to indicate that two image items form a stereo pair suitable for stereoscopic viewing.

§ 6. Brands, Internet media types and file extensions

§ 6.1. Brands overview

As defined by [ISOBMFF], the presence of a brand in the `FileTypeBox` can be interpreted as the permission for those AV1 Image File Format readers/parsers and AV1 Image File Format renderers that only implement the features required by the brand, to process the corresponding file and only the parts (e.g. items or sequences) that comply with the brand.

An AV1 Image File Format file may conform to multiple brands. Similarly, an AV1 Image File Format reader/parser or AV1 Image File Format renderer may be capable of processing the features associated with one or more brands.

If any of the brands defined in this document is specified in the `major_brand` field of the `FileTypeBox`, the file extension and Internet Media Type should respectively be "`.avif`" and "`image/avif`" as defined in §10 AVIF Media Type Registration.

§ 6.2. AVIF image and image collection brand

The brand to identify AV1 image items is ***avif***.

Files that indicate this brand in the `FileTypeBox` shall comply with the following:

- The primary image item shall be an AV1 Image Item or be a derived image that references directly or indirectly one or more items that all are AV1 Image Items.
- AV1 auxiliary image items may be present in the file.

Files that conform with these constraints should include the brand `avif` in the `FileTypeBox`.

Additionally, the brand ***avio*** is defined. If the file indicates the brand `avio` in the `FileTypeBox`, then the primary image item or all the items referenced by the primary image item shall be AV1 image items made only of Intra Frames.

§ 6.3. AVIF image sequence brands

The brand to identify AV1 image sequences is ***avis***.

Files that indicate this brand in the `FileTypeBox` shall comply with the following:

- they shall contain one or more AV1 image sequences.
- they may contain AV1 auxiliary image sequences.

Files that conform with these constraints should include the brand `avis` in the `FileTypeBox`.

Additionally, if a file contains AV1 image sequences and the brand `avio` is used in the `FileTypeBox`, the item constraints for this brand shall be met and at least one of the AV1 image sequences shall be made only of AV1 Samples marked as '`sync`'. Conversely, if such a track exists and the constraints of the brand `avio` on AV1 image items are met, the brand should be used.

> NOTE: As defined in [MIAF], a file that is primarily an image sequence still has at least an image item. Hence, it can also declare brands for signaling the image item.

§ 7. General constraints

The following constraints are common to files compliant with this specification:

- The file shall be compliant with the [MIAF] specification and list '`miaf`' in the `FileTypeBox`.
- The file shall list '`avif`' or '`avis`' in the `FileTypeBox`.
- Transformative properties shall not be associated with items in a derivation chain (as defined in [MIAF]) that serves as an input to a grid derived image item. For example, if a file contains a grid item and its referenced coded image items, cropping, mirroring or rotation transformations are only permitted on the grid item itself.

  > NOTE: This constraint further restricts files compared to [MIAF].

§ 8. Profiles

§ 8.1. Overview

The profiles defined in this section are for enabling interoperability between AV1 Image File Format files and AV1 Image File Format readers/parsers. A profile imposes a set of specific restrictions and is signaled by brands defined in this specification.

The `FileTypeBox` should declare at least one profile that enables decoding of the primary image item. It is not an error for the encoder to include an auxiliary image that is not allowed by the specified profile(s). If '`avis`' is declared in the `FileTypeBox` and a profile is declared in the `FileTypeBox`, the profile shall also enable decoding of at least one image sequence track. The profile should allow decoding of any associated auxiliary image sequence tracks, unless it is acceptable to decode the image sequence track without its auxiliary image sequence tracks.

It is possible for a file compliant to this AV1 Image File Format to not be able to declare an AVIF profile, if the corresponding AV1 encoding characteristics do not match any of the defined profiles.

> NOTE: [AV1] supports 3 bit depths: 8, 10 and 12 bits, and the maximum dimensions of a coded image is 65536x65536, when `seq_level_idx` is set to 31 (maximum parameters level).

> EXAMPLE 2 ¶
> If an image is encoded with dimensions (respectively a bit depth) that exceed the maximum dimensions (respectively bit depth) required by the AV1 profile and level of the AVIF profiles defined in this specification, the file will only signal general AVIF brands.

§ 8.2. ***AVIF Baseline Profile***

This section defines the MIAF AV1 Baseline profile of [HEIF], specifically for [AV1] bitstreams, based on the constraints specified in [MIAF] and identified by the brand ***MA1B***.

If the brand '`MA1B`' is in the `FileTypeBox`, the common constraints in the section §6 Brands, Internet media types and file extensions shall apply.

The following shared conditions and requirements from [MIAF] shall apply:

- self-containment (subclause 8.2)

The following shared conditions and requirements from [MIAF] should apply:

- grid-limit (subclause 8.4)
- single-track (subclause 8.5)
- edit-lists (subclause 8.6)
- matched-duration (subclause 8.7)

The following additional constraints apply to all AV1 Image Items and all AV1 Image Sequences:

- The AV1 profile shall be the Main Profile and the level shall be 5.1 or lower.

  > NOTE: AV1 tiers are not constrained because timing is optional in image sequences and is not relevant in image items or collections.

  > NOTE: Level 5.1 is chosen for the Baseline profile to ensure that no single coded image exceeds 4k resolution, as some decoders may not be able to handle larger images. More precisely, following [AV1] level definitions, coded image items compliant to the AVIF Baseline profile may not have a number of pixels greater than 8912896, a width greater than 8192 or a height greater than 4352. It is still possible to use the Baseline profile to create larger images using a grid derived image item.

> EXAMPLE 3 ¶
> A file containing items compliant with this profile is expected to list the following brands, in any order, in the `FileTypeBox`:
>
> `avif, mif1, miaf, MA1B`
>
> A file containing a '`pict`' track compliant with this profile is expected to list the following brands, in any order, in the `FileTypeBox`:
>
> `avis, msf1, miaf, MA1B`
>
> A file containing a '`pict`' track compliant with this profile and made only of AV1 Samples marked '`sync`' is expected to list the following brands, in any order, in the `FileTypeBox`:
>
> `avis, avio, msf1, miaf, MA1B`

§ 8.3. ***AVIF Advanced Profile***

This section defines the MIAF AV1 Advanced profile of [HEIF], specifically for [AV1] bitstreams, based on the constraints specified in [MIAF] and identified by the brand ***MA1A***.

If the brand '`MA1A`' is in the `FileTypeBox`, the common constraints in the section §6 Brands, Internet media types and file extensions shall apply.

The following shared conditions and requirements from [MIAF] shall apply:

- self-containment (subclause 8.2)

The following shared conditions and requirements from [MIAF] should apply:

- grid-limit (subclause 8.4)
- single-track (subclause 8.5)
- edit-lists (subclause 8.6)
- matched-duration (subclause 8.7)

The following additional constraints apply to all AV1 Image Items:

- The AV1 profile shall be the High Profile and the level shall be 6.0 or lower.

  > NOTE: Following [AV1] level definitions, coded image items compliant to the AVIF Advanced profile may not have a number of pixels greater than 35651584, a width greater than 16384 or a height greater than 8704. It is still possible to use the Advanced profile to create larger images using a grid derived image item.

The following additional constraints apply only to AV1 Image Sequences:

- The AV1 profile shall be either Main Profile or High Profile.
- The AV1 level for Main Profile shall be 5.1 or lower.
- The AV1 level for High Profile shall be 5.1 or lower.

> EXAMPLE 4 ¶
> A file containing items compliant with this profile is expected to list the following brands, in any order, in the `FileTypeBox`:
>
> `avif, mif1, miaf, MA1A`
>
> A file containing a '`pict`' track compliant with this profile is expected to list the following brands, in any order, in the `FileTypeBox`:
>
> `avis, msf1, miaf, MA1A`

§ 9. Box requirements

§ 9.1. Image item boxes

This section discusses the box requirements for an AVIF file containing image items.

§ 9.1.1. Minimum set of boxes

As indicated in §7 General constraints, an AVIF file is a compliant [MIAF] file. As a consequence, some [ISOBMFF] or [HEIF] boxes are required, as indicated in the following table. The order of the boxes is indicative in the table. The specifications listed in the "Specification" column may require a specific order for a box or for its children and the order shall be respected. For example, per [ISOBMFF], the `FileTypeBox` is required to appear first in an AVIF file. The "Version(s)" column in the following table lists the version(s) of the boxes allowed by this brand. With the exception of item properties marked as non-essential, other versions of the boxes shall not be used. "-" means that the box does not have a version.

Top-Level	Level 1	Level 2	Level 3	Version(s)	Specification	Note
ftyp				-	[ISOBMFF]	
meta				0	[ISOBMFF]	
	hdlr			0	[ISOBMFF]	
	pitm			0, 1	[ISOBMFF]	
	iloc			0, 1, 2	[ISOBMFF]	
	iinf			0, 1	[ISOBMFF]	
		infe		2, 3	[ISOBMFF]	
	iprp			-	[ISOBMFF]	
		ipco		-	[ISOBMFF]	
			av1C	-	AVIF	
			ispe	0	[HEIF]	
			pixi	0	[HEIF]	
		ipma		0, 1	[ISOBMFF]	
mdat				-	[ISOBMFF]	The coded payload may be placed in '`idat`' rather than '`mdat`', in which case '`mdat`' is not required.

§ 9.1.2. Requirements on additional image item related boxes

The boxes indicated in the following table may be present in an AVIF file to provide additional signaling for image items. If present, the boxes shall use the version indicated in the table unless the box is an item property marked as non-essential. AVIF readers are expected to understand the boxes and versions listed in this table. The order of the boxes in the table may not be the order of the boxes in the file. Specifications may require a specific order for a box or for its children and the order shall be respected. Additionally, the '`free`' and '`skip`' boxes may be present at any level in the hierarchy and AVIF readers are expected to ignore them. Additional boxes in the '`meta`' hierarchy not listed in the following table may also be present and may be ignored by AVIF readers.

Top-Level	Level 1	Level 2	Level 3	Version(s)	Specification	Description
meta						See §9.1.1 Minimum set of boxes
	dinf			-	[ISOBMFF]	Used to indicate the location of the media information
		dref		0	[ISOBMFF]	
	iref			0, 1	[ISOBMFF]	Used to indicate directional relationships between images or metadata
		auxl		-	[HEIF]	Used when an image is auxiliary to another image
		thmb		-	[HEIF]	Used when an image is a thumbnail of another image
		dimg		-	[HEIF]	Used when an image is derived from another image
		prem		-	[HEIF]	Used when the color values in an image have been premultiplied with alpha values
		cdsc		-	[HEIF]	Used to link metadata with an image
	idat			-	[ISOBMFF]	Typically used to store derived image definitions or small pieces of metadata
	grpl			-	[ISOBMFF]	Used to indicate that multiple images are semantically grouped
		altr		0	[ISOBMFF]	Used when images in a group are alternatives to each other
		ster		0	[HEIF]	Used when images in a group form a stereo pair
	iprp					See §9.1.1 Minimum set of boxes
		ipco				See §9.1.1 Minimum set of boxes
			pasp	-	[ISOBMFF]	Used to signal pixel aspect ratio. If present, shall indicate a pixel aspect ratio of 1:1
			colr	-	[ISOBMFF]	Used to signal color information such as color primaries
			auxC	0	[HEIF]	Used to signal the type of an auxiliary image (e.g. alpha, depth)
			clap	-	[ISOBMFF]	Used to signal cropping applied to an image
			irot	-	[HEIF]	Used to signal a rotation applied to an image
			imir	-	[HEIF]	Used to signal a mirroring applied to an image
			clli	-	[ISOBMFF]	Used to signal HDR content light level information for an image
			cclv	-	[ISOBMFF]	Used to signal HDR content color volume for an image
			mdcv	-	[ISOBMFF]	Used to signal HDR mastering display color volume for an image
			amve	-	[ISOBMFF]	Used to signal the nominal ambient viewing environment for the display of the content
			reve	0	[HEIF]	Used to signal the viewing environment in which the image was mastered
			ndwt	0	[HEIF]	Used to signal the nominal diffuse white luminance of the content
			a1op	-	AVIF	Used to configure which operating point to select when there are multiple choices
			lsel	-	[HEIF]	Used to configure rendering of a multilayered image
			a1lx	-	AVIF	Used to assist reader in parsing a multilayered image
			cmin	0	[HEIF]	Used to signal the camera intrinsic matrix
			cmex	0	[HEIF]	Used to signal the camera extrinsic matrix

§ 10. AVIF Media Type Registration

The media type "`image/avif`" is officially registered with IANA and available at: https://www.iana.org/assignments/media-types/image/avif.

§ 11. Changes since v1.1.0 release

- EDITORIAL: Stop using `dfn value` for definitions.
- EDITORIAL: Add assert-ids in the spec for conformance file testing and ComplianceWarden
- Add required list of boxes for AVIF files
- EDITORIAL: Add "per item" to item property definitions
- EDITORIAL: Fix broken link for latest-draft.html
- Relax constraint on transformative properties in derivation chains to only apply to grid items
- Change ispe width to correspond to UpscaledWidth
- Clarify relationship between av1C, metadata OBUs and item properties
- EDITORIAL: Update list of other item properties
- Further clarify relationship between av1C, metadata OBUs and item properties
- Add information on tmap, grpl and altr
- Replace recommendations regarding still picture flags in image items by a note
- Add section 4.2.2 "Sample Transform Derived Image Item"
- Add Appendix A "Sample Transform Derived Image Item Examples"
- Add restriction on usage of clap property
- Adopt MIAF shared constraints
- EDITORIAL: Clean up usage of dfn and linking
- Clarify required versions of non-essential item properties
- EDITORIAL: Add refs, fix wording and format in clauses 1,2
- EDITORIAL: Add sato, alpha, depth, progressive in Scope
- Clean up linking to base specs and duplicated information
- EDITORIAL: Use Markdown syntax coloring for code snippets
- EDITORIAL: Consistently use dfn noexport for derived items
- EDITORIAL: Remove Sample Transform sections from TOC
- EDITORIAL: Add references in the Sample Transform sections
- EDITORIAL: Indent notes as the list items they refer to
- EDITORIAL: Remove inconsistent dots in 9.1.2
- Change structure of optional table of boxes
- Add hidden image item recommendation
- Remove mentions of ftyp compatible_brands
- Remove avio brand recommendation
- EDITORIAL: Fix broken lines in Sample Transform examples
- EDITORIAL: Make assert IDs between profiles unique
- Update the HEIF, ISOBMFF, and MIAF references to the latest versions
- EDITORIAL: Merge asserts for avio brand
- EDITORIAL: Clarify `sato` item requirements

§ Appendix A: (informative) Sample Transform Derived Image Item Examples

This informative appendix contains example recipes for extending base AVIF features with Sample Transform Derived Image Items.

§ Bit depth extension

Sample Transform Derived Image Items allow for more than 12 bits per channel per sample by combining several AV1 image items in multiple ways.

§ Suffix bit depth extension

The following example describes how to leverage a Sample Transform Derived Image Item on top of a regular 8-bit MIAF image item to extend the decoded bit depth to 16 bits.

Consider the following:

- A MIAF image item being a losslessly coded image item,
  and its `PixelInformationProperty` with `bits_per_channel`=8,
- Another image item being a lossily or losslessly coded image item with the same spatial dimensions, the same number of channels, and the same chroma subsampling (or lack thereof) as the first input image item,
  and its `PixelInformationProperty` with `bits_per_channel`=8,
- A Sample Transform Derived Image Item with the two items above as input in this order,
  and its `PixelInformationProperty` with `bits_per_channel`=16,
  and the following `SampleTransform` fields:
  - `version`=0
  - `bit_depth`=2 (signed 32-bit `constant`s, stack values and intermediate results)
  - `token_count`=5
    - `token`=0, `constant`=256
    - `token`=1 (sample from 1st input image item)
    - `token`=130 (product)
    - `token`=2 (sample from 2nd input image item)
    - `token`=128 (sum)

This is equivalent to the following postfix notation (parentheses added for clarity):

$$\text{sample}_{\text{output}} = \left(256\ \ \text{sample}_1\ \times\right)\ \text{sample}_2\ +$$

This is equivalent to the following infix notation:

$$\text{sample}_{\text{output}} = 256 \times \text{sample}_1 + \text{sample}_2$$

Each output sample is equal to the sum of a sample of the first input image item shifted to the left by 8 bits and of a sample of the second input image item. This can be viewed as a bit depth extension of the first input image item by the second input image item. The first input image item contains the 8 most significant bits and the second input image item contains the 8 least significant bits of the 16-bit output reconstructed image item. It is impossible to achieve a bit depth of 16 with a single AV1 image item.

> NOTE: If the first input image item is the primary image item and is enclosed in an '`altr`' group (see §5.1 'altr' group) with the Sample Transform Derived Image Item, the first input image item is also a backward-compatible 8-bit regular coded image item that can be used by readers that do not support Sample Transform Derived Image Items or do not need extra precision.

> NOTE: The second input image item can be marked as hidden to prevent readers from surfacing it to users.

> NOTE: The second input image item loses its meaning of the least significant part if any of the most significant bits changes, so the first input image item has to be losslessly encoded. The second input image item supports reasonable loss during encoding.

> NOTE: This pattern can be used for reconstructed bit depths beyond 16 by combining more than two input image items or with various input bit depth configurations and operations.

§ Residual bit depth extension

The following example describes how to leverage a Sample Transform Derived Image Item on top of a regular 12-bit MIAF image item to extend the decoded bit depth to 16 bits.
It differs from the Suffix bit depth extension by its slightly longer series of operations allowing its first input image item to be lossily encoded.

Consider the following:

- A MIAF image item being a lossily coded image item,
  and its `PixelInformationProperty` with `bits_per_channel`=12,
- Another image item being a lossily or losslessly coded image item with the same spatial dimensions, the same number of channels, and the same chroma subsampling (or lack thereof) as the first input image item,
  and its `PixelInformationProperty` with `bits_per_channel`=8,
  with the following contraints:

  For each sample position in each plane,
  $\text{sample}_{\text{original}}$ being the value of the 16-bit original sample at that position in that plane,
  sample_1 being the value of the 12-bit sample of the first input image at that position in that plane,
  sample_2 being the value of the sample of the second input image at that position in that plane,
  $\approx$ representing similarity within compression loss range,
  - $\text{sample}_1 \approx \frac{\text{sample}_{\text{original}}}{2^4}$
  - $\text{sample}_2 \approx \text{sample}_{\text{original}} - 2^4 \times \text{sample}_1 + 2^7$
  - $0 \leq \text{sample}_1 < 2^{12}$
  - $0 \leq \text{sample}_2 < 2^8$
  - $0 \leq 2^4 \times \text{sample}_1 + \text{sample}_2 - 2^7 < 2^{16}$

  > NOTE: Files that do not respect this constraint will still decode successfully because Clause §4.2.3.1 Definition mandates the resulting values to be each clamped to fit in the number of bits per sample as defined by the `PixelInformationProperty` of the reconstructed image item.
- A Sample Transform Derived Image Item with the two items above as input in this order,
  and its `PixelInformationProperty` with `bits_per_channel`=16,
  and the following `SampleTransform` fields:
  - `version`=0
  - `bit_depth`=2 (signed 32-bit `constant`s, stack values and intermediate results)
  - `token_count`=7
    - `token`=0, `constant`=16
    - `token`=1 (sample from 1st input image item)
    - `token`=130 (product)
    - `token`=2 (sample from 2nd input image item)
    - `token`=128 (sum)
    - `token`=0, `constant`=128
    - `token`=129 (difference)

This is equivalent to the following postfix notation (parentheses added for clarity):

$$\text{sample}_{\text{output}} = \left(\left(16\ \ \text{sample}_1\ \times\right)\ \text{sample}_2\ +\right)\ 128\ -$$

This is equivalent to the following infix notation:

$$\text{sample}_{\text{output}} = 16 \times \text{sample}_1 + \text{sample}_2 - 128$$

Each output sample is equal to the sum of a sample of the first input image item shifted to the left by 4 bits and of a sample of the second input image item offset by -128. This can be viewed as a bit depth extension of the first input image item by the second input image item, which contains the residuals to correct the precision loss of the first input image item.

> NOTE: If the first input image item is the primary image item and is enclosed in an '`altr`' group (see §5.1 'altr' group) with the derived image item, the first input image item is also a backward-compatible 12-bit regular coded image item that can be used by decoding contexts that do not support Sample Transform Derived Image Items or do not need extra precision.

NOTE: The second input image item can be marked as hidden to prevent readers from surfacing it to users.

NOTE: The first input image item supports reasonable loss during encoding because the second input image item "overlaps" by 4 bits to correct the loss. The second input image item supports reasonable loss during encoding.

NOTE: This pattern can be used for reconstructed bit depths beyond 16 by combining more than two input image items or with various input bit depth configurations and operations.

§ Conformance

Conformance requirements are expressed with a combination of descriptive assertions and RFC 2119 terminology. The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in the normative parts of this document are to be interpreted as described in RFC 2119. However, for readability, these words do not appear in all uppercase letters in this specification.

All of the text of this specification is normative except sections explicitly marked as non-normative, examples, and notes. [RFC2119]

Examples in this specification are introduced with the words “for example” or are set apart from the normative text with `class="example"`, like this:

> EXAMPLE 5 ¶
> This is an example of an informative example.

Informative notes begin with the word “Note” and are set apart from the normative text with `class="note"`, like this:

Note, this is an informative note.

§ Index

§ Terms defined by this specification

- a1lx
- a1op
- av01
- AV1 Alpha Image Item
- AV1 Alpha Image Sequence
- AV1 Auxiliary Image Item
- AV1 Auxiliary Image Sequence
- av1C
- AV1 Depth Image Item
- AV1 Depth Image Sequence
- AV1 Image File Format
- AV1 Image Item
- AV1 Image Item Data
- AV1 Image Sequence
- AV1ItemConfigurationProperty
- AV1LayeredImageIndexingProperty
- AVIF
- avif
- AVIF Advanced Profile
- AVIF Baseline Profile
- AVIF file
- avio
- avis
- bit_depth
- constant
- expression
- grid derived image item
- intermediate bit depth
- layer_size
- MA1A
- MA1B
- num_bits
- operand
- OperatingPointSelectorProperty
- operator
- op_index
- reserved
- SampleTransform
- Sample Transform Derived Image Item
- sato
- token
- token_count
- tone map derived image item
- version

§ Terms defined by reference

- [AV1] defines the following terms:
  - AV1 bitstream
  - color_range
  - FrameHeight
  - Intra Frame
  - max_frame_height_minus1
  - max_frame_width_minus1
  - Metadata OBU
  - mono_chrome
  - Operating Point
  - operating_points_cnt_minus_1
  - reduced_still_picture_header
  - render_height_minus1
  - render_width_minus1
  - seq_level_idx
  - Sequence Header OBU
  - spatial_id
  - still_picture
  - Temporal Unit
  - UpscaledWidth
- [AV1-ISOBMFF] defines the following terms:
  - AV1 Sample
  - AV1 Track
  - AV1CodecConfigurationBox
- [HEIF] defines the following terms:
  - aux_type
  - auxC
  - AuxiliaryTypeInfoBox
  - AuxiliaryTypeProperty
  - auxl
  - bits_per_channel
  - cdsc
  - cmex
  - cmin
  - derived image item
  - dimg
  - grid
  - hidden image item
  - image_height
  - image_width
  - imir
  - irot
  - ispe
  - layer_id
  - lsel
  - ndwt
  - pict
  - PixelInformationProperty
  - pixi
  - prem
  - reve
  - ster
  - thmb
  - tmap
- [ISOBMFF] defines the following terms:
  - altr
  - amve
  - cclv
  - clap
  - clli
  - colour_type
  - ColourInformationBox
  - colr
  - ContentLightLevelBox
  - dinf
  - dref
  - FileTypeBox
  - free
  - from_item_ID
  - ftyp
  - full_range_flag
  - GroupsListBox
  - grpl
  - hdlr
  - idat
  - iinf
  - iloc
  - infe
  - ipco
  - ipma
  - iprp
  - iref
  - ItemReferenceBox
  - major_brand
  - MasteringDisplayColourVolumeBox
  - matrix_coefficients
  - mdat
  - mdcv
  - meta
  - nclx
  - pasp
  - pitm
  - reference_count
  - SingleItemTypeReferenceBox
  - SingleItemTypeReferenceBoxLarge
  - skip
  - sync
  - to_item_ID
- [MIAF] defines the following terms:
  - edit-lists
  - grid-limit
  - matched-duration
  - miaf
  - MIAF auxiliary image item
  - MIAF auxiliary image sequence
  - MIAF image item
  - MIAF image sequence
  - primary image item
  - self-containment
  - single-track

§ References

§ Normative References

[AV1]
AV1 Bitstream & Decoding Process Specification. LS. URL: https://aomediacodec.github.io/av1-spec/av1-spec.pdf

[AV1-ISOBMFF]
AV1 Codec ISO Media File Format Binding. LS. URL: https://aomediacodec.github.io/av1-isobmff/

[CICP]
H.273 : Coding-independent code points for video signal type identification. International Standard. URL: https://www.itu.int/rec/T-REC-H.273

[HEIF]
Information technology — High efficiency coding and media delivery in heterogeneous environments — Part 12: Image File Format. International Standard. URL: https://www.iso.org/standard/89044.html

[ISOBMFF]
Information technology — Coding of audio-visual objects — Part 12: ISO base media file format. International Standard. URL: https://www.iso.org/standard/83102.html

[MIAF]
Information technology -- Multimedia application format (MPEG-A) -- Part 22: Multi-image application format (MIAF). Enquiry. URL: https://www.iso.org/standard/74417.html

[RFC2119]
S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. March 1997. Best Current Practice. URL: https://datatracker.ietf.org/doc/html/rfc2119